

Learning AI Training Guide

The Learning AI uses Deep Q-Learning (DQN) with PyTorch to learn optimal gin rummy play through self-play and curriculum training.

Quick Start

```
# Install learning dependencies
uv sync --extra learning

# Train a model (default: 10,000 episodes)
uv run gin-train --episodes 10000 --output models/learning_ai.pt

# Use trained model in simulation
uv run gin-simulate --ai1-type learning --ai1-model models/learning_ai.pt --ai2-type context -n 100
```

Architecture

Three Separate Networks

The AI uses three specialized neural networks, one for each decision type:

Network	Decision	Input Size	Output	Hidden Layers
DrawNet	Deck vs discard	198	2 Q-values	[128, 64]
DiscardNet	Which card to discard	250	11 Q-values	[256, 128]
KnockNet	Whether to knock	198	2 Q-values	[64, 32]

State Representation (~200 features)

The game state is encoded as a fixed-size tensor:

State Vector Components:

```
+++ Hand (52)          - One-hot: which cards are in hand
+++ Dead cards (52)     - Multi-hot: cards known unavailable
+++ Discard top (52)    - One-hot: top card of discard pile
+++ Opponent patterns:
|  +++ Pickup ranks (13) - Normalized counts
|  +++ Pickup suits (4)  - Normalized counts
|  +++ Discard ranks (13) - Normalized counts
|  +++ Discard suits (4) - Normalized counts
+++ Game features (8):
|  +++ deck_position_pct - 0.0 (full) to 1.0 (empty)
|  +++ my_deadwood_norm  - deadwood / 100
|  +++ can_knock         - 1.0 if deadwood <= 10
|  +++ is_gin            - 1.0 if deadwood == 0
|  +++ score_diff_norm   - (my_score - opp_score) / 100
|  +++ points_to_win_norm - points_to_win / 100
|  +++ meld_count_norm   - num_melds / 4
|  +++ live_outs_norm    - live_outs / 20
+++ Drawn card (52)     - One-hot (discard decision only)
```

Training Process

Deep Q-Learning (DQN)

The training uses standard DQN with:

1. **Experience Replay:** Stores (state, action, reward, next_state, done) tuples in a buffer, samples random batches for training to break correlation between consecutive experiences.
2. **Target Network:** A separate copy of the networks updated periodically (every 100 episodes) to stabilize training.
3. **Epsilon-Greedy Exploration:** Starts with 100% random actions, decays to 5% over training.

Reward Function

End of Round Rewards: | Outcome | Reward | |———|———| | Win by gin | +50 | | Win by knock | +20 + points/5 | | Win by undercut | +30 + points/5 | | Loss | -points/5 | | Draw | 0 |

Intermediate Rewards (per turn): - Deadwood reduction: +0.1 per point reduced - Meld completion: +1.0 per new meld - Key out pickup: +0.5 for meld-completing cards

Curriculum Learning

Training progresses through stages with increasingly difficult opponents:

Stage	Opponent	Episodes	Purpose
1	BasicAI	5,000	Learn fundamentals
2	ContextAwareAI	5,000	Learn advanced play
3	Self-play	10,000	Refine strategy

CLI Options

gin-train

```
uv run gin-train [OPTIONS]
```

Options:

```
--episodes N      Number of training episodes (default: 10000)
--output PATH     Path to save model (default: models/learning_ai.pt)
--tensorboard DIR TensorBoard log directory (default: runs/gin_learning)
--resume PATH     Resume training from checkpoint
--eval-freq N     Evaluate every N episodes (default: 500)
--save-freq N     Save checkpoint every N episodes (default: 1000)
-v, --verbose     Enable verbose logging
```

gin-simulate with LearningAI

```
uv run gin-simulate \
  --ai1-type learning \
  --ai1-model models/learning_ai.pt \
  --ai2-type context \
  -n 100
```

Monitoring Training

TensorBoard

```
# In a separate terminal
tensorboard --logdir runs/
```

```
# Open http://localhost:6006 in browser
```

Metrics tracked: - **reward/episode** - Total reward per episode - **exploration_rate** - Current epsilon value - **eval/win_rate** - Win rate against BasicAI (evaluated periodically)

Console Output

Training prints progress every 100 episodes:

```
Episode 100: reward=12.3, exploration=0.951, buffer=1523
Episode 200: reward=-5.2, exploration=0.904, buffer=3102
...
Episode 500: win_rate=0.42, avg_points=45.2, exploration=0.779
```

Configuration

Training hyperparameters are in `config/learning.toml`:

```
[learning]
num_episodes = 10000
batch_size = 64
learning_rate = 0.001
gamma = 0.99          # Discount factor

exploration_start = 1.0
exploration_end = 0.05
exploration_decay = 0.9995

target_update_freq = 100 # Sync target network every N episodes
buffer_capacity = 100000
min_buffer_size = 1000   # Start training after N experiences

[learning.rewards]
win_by_gin = 50.0
win_by_knock = 20.0
deadwood_reduction_bonus = 0.1
meld_completion_bonus = 1.0
```

Model Files

Checkpoints are saved to `models/` directory:

```
models/
+-- learning_ai.pt    # Contains all three networks + metadata
```

Metadata includes: - **episode** - Training episode when saved - **exploration_rate** - Current epsilon - **curriculum_idx** - Current curriculum stage

Tips for Training

1. **Start with fewer episodes** to verify setup works:

```
uv run gin-train --episodes 100 --eval-freq 50
```

2. **Monitor early win rate** - should improve from ~30% to 50%+ against BasicAI within first 2000 episodes.

3. **Resume training** if interrupted:

```
uv run gin-train --resume models/learning_ai.pt --episodes 5000
```

4. Compare to baseline after training:

```
# Learning vs Context
```

```
uv run gin-simulate --ai1-type learning --ai1-model models/learning_ai.pt --ai2-type context -n 100
```

```
# Learning vs Basic
```

```
uv run gin-simulate --ai1-type learning --ai1-model models/learning_ai.pt --ai2-type basic -n 100
```

Expected Results

After full training (20,000 episodes): - Win rate vs BasicAI: Target >60% - Win rate vs ContextAwareAI: Target >55%

Note: An untrained LearningAI (random networks) will lose nearly every game. Training is required before the model is competitive.